Mobile SDK Implementation Guide

Welcome to the Deluxe Mobile SDK Implementation Guide! This guide provides developers with clear, step-by-step instructions for integrating the Deluxe Mobile Payments SDK. It covers setup, configuration, and implementation of key SDK features to help you get up and running quickly and efficiently.

Prerequisites

Before you begin integrating the Deluxe POS SDKs, ensure you have the following prerequisites ready:

- **Merchant Client Application Credentials and Access Token:** Ensure that merchant onboarding is complete and that you've received your client application credentials and access token before proceeding.
- **Access to Deluxe Developer Portal:** This portal provides additional resources, API documentation, and integration tools.

Note: *If you don't have a Deluxe Merchant Account or Access Token, simply visit Merchant Services Get Started and contact us today.*

SDK Specifications

iOS Specific Requirements:

- Minimum iOS Version 13.0
- Maximum iOS Version - Latest iOS
- Supported iPhone Models: All iPhones running iOS 13 or later, from iPhone 7 up to the latest iPhone models.

Android Specific Requirements:

- Minimum SDK Version 24
- Maximum SDK Version 36

ON THIS PAGE

Prerequisites
SDK Specifications
SDK Integration Steps
Environment Configuration
Terminal Management
Transaction Management
Merchant Operation Services (Non-Transaction Services)
SDK Logger Usage
Additional Help & Support

SDK Integration Steps

Download the SDK:

To obtain the latest version of the SDK, please contact isvinquiries@deluxe.com.

iOS Instructions:

- Download and extract all the XCFrameworks.
- Open your Xcode project.
- Drag and drop the following .xcframework files into your project:
  - PosSDK.xcframework
  - PaymentSDK.xcframework
  - ServiceSDK.xcframework
  - PaymentHostModule.xcframework
  - TapToPaySDK.xcframework
- Ensure the minimum deployment target is set to iOS 13.0 or above.
- Add all five frameworks in the "Link Binary With Libraries" section of your Build Phases.
- Confirm that all five frameworks are added in the "Embed Framework" section of your Build Phases.

Android instructions:

- Create a libs/ folder in your project directory if it doesn't already exist.
- Copy all the .aar files into the libs/ folder
- Open the build.gradle file of your app module.
- Add the following in the repositories
  - gradle repositories { flatDir { dirs 'libs' } }
- Add the following in the dependencies
  - gradle implementation(name: 'possdk-release.aar', ext: 'aar')
  - implementation(name: 'payment-release.aar', ext: 'aar')
  - implementation(name: 'servicesdk-release.aar', ext: 'aar')
  - implementation(name: 'payment-hostmodule-release.aar', ext: 'aar')
  - implementation(name: ‘roamreaderunifiedapi2.4.9.3-release.aar')

Environment Configuration

Before implementing any SDK features in your application, ensure that the POSService singleton instance is properly configured for the required environments.

The SDK currently supports two environments: Validation and Production. By default, it is configured for Validation, which serves as a test environment for development purposes.

iOS instructions:

The POSService class provides a shared instance (POSService.shared) that can be configured using the configure method. This configuration determines which environment (Validation or Production) the SDK will use going forward.

iOS Example:

```
POSService.shared.configure(environment:
.production)
```

The configure method should be called once during app startup according to your application logic.

Android Instructions:

The POSService class can be configured using the configure method. This configuration determines which environment (Validation or Production) the SDK will use going forward.

Android Example:

```
PosService.configure(applicationContext,DeluxeEnvir
onment.PRODUCTION)
```

The configure method should be called once during app startup according to your application logic.

Terminal Management

The TerminalManager class provides core functionality for managing terminal connections and utilities. It is a general-

Currently, TerminalManager supports the following device types:

- **Ingenico Moby5500** – Available on both iOS and Android SDKs.
- **Apple Tap to Pay** – Supported only on iOS SDK (iOS 17 and later) and compatible iPhone devices.

1.) Terminal Connection

iOS instructions:

- Ingenico Moby5500 - iOS Terminal Connection Guide - Ingenico Moby5500
- Apple Tap to Pay - iOS Terminal Connection Guide - Apple Tap to Pay

Android instructions:

- Ingenico Moby5500 - Android Terminal Connection Guide

2.) Terminal Utility Services

This feature is currently available for Ingenico Moby5500.

iOS instructions:

- iOS Terminal Management Guide

Android instructions:

- Android Terminal Management Guide

Transaction Management

The TransactionManager class handles all payment transaction operations. It serves as a general-purpose utility for executing transactions on any supported terminal device.

iOS instructions:

- iOS Transaction Guide

Android instructions:

- Android Transaction Guide

Beyond transaction processing, the SDK provides a suite of services that support merchant operations, offering valuable insights and management capabilities.

Here are some of the key features:

Merchant Authorization Service: This service is responsible for both validating the client API credentials and fetching the merchant authentication token for the merchant.

iOS instructions:

- iOS Merchant Authorization & Accounts Guide

Android instructions:

- Android Merchant Authorization & Accounts Guide

Merchant Account Service: This service retrieves all merchant accounts, each containing details such as company ID, merchant ID (MID), terminal ID (TID), bank ID, DBA (Doing Business As) name, account ID, and more.

iOS instructions:

- iOS Merchant Authorization & Accounts Guide

Android instructions:

- Android Merchant Authorization & Accounts Guide

Transaction Service: The Transaction Services Guide provides a comprehensive overview of the transaction-related services used for retrieving transaction history, summaries, detailed records, and sending digital receipts.

iOS instructions:

- iOS Transaction Services Guide

Android instructions:

- Android Transaction Services Guide

Sale Tax Service: This service represents the tax information that a merchant can apply to different catalog items. It includes

location and tax regulations.

iOS instructions:

- iOS Sale Tax Service Guide

Android instructions:

- Android Sale Tax Service Guide

Sale Item Category Service: This service represents the categories of sale items that a merchant can create and manage. It includes functionalities for creating, updating, and deleting sale item categories, as well as retrieving specific categories.

iOS instructions:

- iOS Sale Item Category Service Guide

Android instructions:

- Android Sale Item Category Service Guide

Sale Item Service: This service is responsible for managing sale items - related activities for the merchant. It includes functionalities such as creating, updating, and deleting sale items.

iOS instructions:

- iOS Sale Item Service Guide

Android instructions:

- Android Sale Item Service Guide

Sale Discount Service: This service allows merchants to apply discounts to their sales transactions. It includes functionalities for creating, updating, and managing discount rules and categories.

iOS instructions:

- iOS Sale Discount Service Guide

SDK Logger Usage

The SDK includes an internal logging mechanism to capture runtime diagnostics, errors, and crash-related data. This will help to capture the runtime logs and will be useful for the SDK support team to troubleshoot the issue easily.

The LogManager must be initialized using the initialize() method to start the logging process. This should be called once during the app's startup to configure the logging environment.

iOS Instructions:

```
LoggerManager.initialize(
withCustomDirectory: "AppLogs",
isDebugEnabled: true // Set to true if you want to
log actively
)
```

Android Instructions:

```
LogManager.initialize(     context =
applicationContext,
logFileDirectoryPath = "AppLogs",
isDebugEnabled = true // Set to true if you want to
log actively )
```

Additional Help & Support

Please refer to the complete SDK reference documents or contact our support team at isvinquiries@deluxe.com.

- iOS SDK Reference Guide
- Android SDK Reference Guide

deluxe. connect

Payments API

Deluxe.com | Privacy Policy | Do Not Sell My Personal Information

Deluxe Corporation, 801 S Marquette Ave,
Minneapolis, MN 55402.

© 2026 Deluxe Corporation